

Competitive Programming

Section 65 Editorial: Polynomial Algorithms And Recurrences

Competitive Programming Course

2026-08-20

Table of contents

1	Shared NTT implementation	3
1.1	C++ helper	3
1.2	Python helper	5
2	A. Cyclic Agreement	7
2.1	Turning equality into correlations	7
2.2	Trace	7
2.3	Correctness proof	7
2.4	Complexity, pitfalls, and tests	8
2.5	Full C++ solution	8
2.6	Full Python solution	9
3	B. Bounded Sum Product	10
3.1	Build the counting object	10
3.2	Why balanced multiplication matters	10
3.3	Correctness proof	10
3.4	Complexity, pitfalls, and tests	11
3.5	Full C++ solution	11
3.6	Full Python solution	11
4	C. Huge Linear Recurrence	13
4.1	Replace matrix exponentiation by polynomial reduction	13
4.2	Correctness proof	13
4.3	Complexity, pitfalls, and tests	13
4.4	Full C++ solution	14
4.5	Full Python solution	15
5	D. Recurrence Recovery	17
5.1	What a discrepancy tells us	17
5.2	Correctness argument	17
5.3	Complexity, pitfalls, and tests	17
5.4	Full C++ solution	18
5.5	Full Python solution	20
6	Stress-testing strategy	22
7	What to carry forward	23

1 Shared NTT implementation

Problems A and B use the same audited transform. `convolution(a,b,keep)` pads to a power of two, transforms both arrays, multiplies pointwise, applies the inverse transform, and optionally truncates the result. The small-factor quadratic branch avoids paying NTT setup costs in the leaves of B's product tree.

1.1 C++ helper

```
#pragma once
#include <algorithm>
#include <cstdint>
#include <vector>

namespace course_ntt {

constexpr int MOD = 998244353;
constexpr int ROOT = 3;

inline int power(long long base, long long exponent) {
    long long result = 1;
    while (exponent > 0) {
        if (exponent & 1) result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return static_cast<int>(result);
}

inline void transform(std::vector<int> &a, bool inverse) {
    int n = static_cast<int>(a.size());
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        while (j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if (i < j) std::swap(a[i], a[j]);
    }

    for (int length = 2; length <= n; length <<= 1) {
        int root = power(ROOT, (MOD - 1) / length);
```

```

        if (inverse) root = power(root, MOD - 2);
        for (int start = 0; start < n; start += length) {
            long long current = 1;
            for (int offset = 0; offset < length / 2; ++offset) {
                int left = a[start + offset];
                int right = static_cast<int>(
                    current * a[start + offset + length / 2] % MOD
                );
                a[start + offset] = left + right;
                if (a[start + offset] >= MOD) a[start + offset] -= MOD;
                a[start + offset + length / 2] = left - right;
                if (a[start + offset + length / 2] < 0)
                    a[start + offset + length / 2] += MOD;
                current = current * root % MOD;
            }
        }
    }

    if (inverse) {
        int inverse_n = power(n, MOD - 2);
        for (int &value : a)
            value = static_cast<int>(1LL * value * inverse_n % MOD);
    }
}

inline std::vector<int> convolution(
    const std::vector<int> &a,
    const std::vector<int> &b,
    int keep = -1
) {
    if (a.empty() || b.empty()) return {};
    int result_size = static_cast<int>(a.size() + b.size() - 1);
    if (keep >= 0) result_size = std::min(result_size, keep);
    if (std::min(a.size(), b.size()) <= 24) {
        std::vector<int> result(result_size);
        for (int i = 0; i < static_cast<int>(a.size()); ++i)
            for (int j = 0; j < static_cast<int>(b.size()) && i + j < result_size; ++j)
                result[i + j] = (result[i + j] + 1LL * a[i] * b[j]) % MOD;
        return result;
    }
    int size = 1;
    while (size < static_cast<int>(a.size() + b.size() - 1)) size <= 1;
    std::vector<int> left(a.begin(), a.end()), right(b.begin(), b.end());
    left.resize(size);
    right.resize(size);
    transform(left, false);
    transform(right, false);
    for (int i = 0; i < size; ++i)
        left[i] = static_cast<int>(1LL * left[i] * right[i] % MOD);
    transform(left, true);
    left.resize(result_size);

```

```

    return left;
}

} // namespace course_ntt

```

1.2 Python helper

```

MOD = 998_244_353
ROOT = 3

def transform(values: list[int], inverse: bool) -> None:
    n = len(values)
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            values[i], values[j] = values[j], values[i]

    length = 2
    while length <= n:
        root = pow(ROOT, (MOD - 1) // length, MOD)
        if inverse:
            root = pow(root, MOD - 2, MOD)
        half = length // 2
        for start in range(0, n, length):
            current = 1
            for offset in range(half):
                left = values[start + offset]
                right = values[start + offset + half] * current % MOD
                values[start + offset] = (left + right) % MOD
                values[start + offset + half] = (left - right) % MOD
                current = current * root % MOD
            length <<= 1

    if inverse:
        inverse_n = pow(n, MOD - 2, MOD)
        for i, value in enumerate(values):
            values[i] = value * inverse_n % MOD

def convolution(
    left: list[int], right: list[int], keep: int | None = None
) -> list[int]:

```

```

if not left or not right:
    return []
result_size = len(left) + len(right) - 1
if keep is not None:
    result_size = min(result_size, keep)
if min(len(left), len(right)) <= 24:
    result = [0] * result_size
    for i, a in enumerate(left):
        for j, b in enumerate(right[: result_size - i]):
            result[i + j] = (result[i + j] + a * b) % MOD
    return result

size = 1 << (len(left) + len(right) - 2).bit_length()
a = left + [0] * (size - len(left))
b = right + [0] * (size - len(right))
transform(a, False)
transform(b, False)
for i in range(size):
    a[i] = a[i] * b[i] % MOD
transform(a, True)
return a[:result_size]

```

2 A. Cyclic Agreement

2.1 Turning equality into correlations

Doing all n comparisons for all n shifts is quadratic. Encode the four letters by two bits. Replace bit zero by sign $+1$ and bit one by sign -1 . For two letters with sign pairs (x,y) and (X,Y) , equality is

$$(1 + xX + yY + xyXY) / 4.$$

If both bits agree, all three products are $+1$, giving one. If at least one bit differs, the four signed terms cancel to zero. Consequently, the score for a shift is n plus three sign correlations, divided by four.

For one correlation, reverse the s feature and duplicate the t feature. Coefficient $n-1+r$ of their convolution is

$$\text{sum}_i \text{feature}_s[i] * \text{feature}_t[(i+r) \bmod n].$$

This equation is worth deriving: it prevents the common reversed-shift bug.

2.2 Trace

For the sample, shift three compares

```
s:           A C G T
t[(i+3) mod 4]: A C G T
```

so all four equality indicators are one. No score can exceed four, hence the answer is 4 3.

2.3 Correctness proof

Lemma 1. The three-feature formula is one exactly for equal letters.

Proof. It is the product of the two bit-equality indicators $(1+xX)/2$ and $(1+yY)/2$. Expanding their product gives the formula. QED.

Lemma 2. The read convolution coefficient is the feature correlation at shift r .

Proof. Reversed position $n-1-i$ pairs at coefficient $n-1+r$ with position $r+i$ of the doubled array, which contains the feature of $t[(r+i) \bmod n]$. Summing over i gives exactly the correlation. QED.

By summing Lemma 1 over all positions and using Lemma 2 for each feature, the computed score is exact. Scanning shifts increasingly and updating only on a strict improvement chooses the smallest optimal shift.

2.4 Complexity, pitfalls, and tests

Three convolutions take $O(n \log n)$ time and $O(n)$ memory. Test length one, all equal letters, no equal letters, and ties between several shifts. Using modular division by four is safe because the modulus is prime and not two.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
#include "../ntt.hpp"
using namespace std;
using course_ntt::MOD;
using course_ntt::convolution;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string s, t;
    if (!(cin >> n >> s >> t)) return 0;

    // Equality of two two-bit symbols is
    //  $(1+X)(1+Y)/4 = (1+X+Y+XY)/4$  for signs  $\pm 1$ .
    const int inverse_four = course_ntt::power(4, MOD - 2);
    const string alphabet = "ACGT";
    vector<int> score(n, n);
    for (int feature = 0; feature < 3; ++feature) {
        vector<int> left(n), doubled_right(2 * n);
        for (int i = 0; i < n; ++i) {
            int a = alphabet.find(s[i]);
            int b = alphabet.find(t[i]);
            int sign_a = feature == 0 ? (a & 1) : feature == 1 ? (a >> 1) : ((a & 1) ^ (a >> 1));
            int sign_b = feature == 0 ? (b & 1) : feature == 1 ? (b >> 1) : ((b & 1) ^ (b >> 1));
            left[n - 1 - i] = sign_a ? MOD - 1 : 1;
            int value = sign_b ? MOD - 1 : 1;
            doubled_right[i] = doubled_right[i + n] = value;
        }
        vector<int> product = convolution(left, doubled_right);
        for (int shift = 0; shift < n; ++shift) {
            score[shift] += product[n - 1 + shift];
            if (score[shift] >= MOD) score[shift] -= MOD;
        }
    }
    int best = -1, best_shift = 0;
    for (int shift = 0; shift < n; ++shift) {
        int matches = static_cast<int>(1LL * score[shift] * inverse_four % MOD);
        if (matches > best) best = matches, best_shift = shift;
    }
}
```

```

    cout << best << ' ' << best_shift << '\n';
}

```

2.6 Full Python solution

```

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
from ntt import MOD, convolution

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    n = int(data[0])
    s, t = data[1].decode(), data[2].decode()
    code = {char: index for index, char in enumerate("ACGT")}
    scores = [n] * n
    for feature in range(3):
        left = [0] * n
        right = [0] * (2 * n)
        for i, (x, y) in enumerate(zip(s, t)):
            a, b = code[x], code[y]
            bit_a = (a & 1) if feature == 0 else (a >> 1) if feature == 1 else ((a & 1) ^ (a >> 1))
            bit_b = (b & 1) if feature == 0 else (b >> 1) if feature == 1 else ((b & 1) ^ (b >> 1))
            left[n - 1 - i] = MOD - 1 if bit_a else 1
            value = MOD - 1 if bit_b else 1
            right[i] = right[i + n] = value
        product = convolution(left, right)
        for shift in range(n):
            scores[shift] = (scores[shift] + product[n - 1 + shift]) % MOD
    inverse_four = pow(4, MOD - 2, MOD)
    matches = [value * inverse_four % MOD for value in scores]
    best = max(matches)
    print(best, matches.index(best))

if __name__ == "__main__":
    main()

```

3 B. Bounded Sum Product

3.1 Build the counting object

Dial i contributes polynomial

$$P_i(x) = 1 + x + \dots + x^{a_i}.$$

When products are expanded, choosing term x^{choice} from each factor records one dial setting. Exponents add, so the coefficient of x^K in the complete product counts exactly the requested settings.

We never need degrees above K . Discarding them after any multiplication is safe: all remaining exponents are nonnegative, so a discarded term can never return to degree K .

3.2 Why balanced multiplication matters

Suppose a long polynomial is multiplied successively by thousands of short ones. Its large prefix is transformed repeatedly. A min-heap always combines the two shortest current factors, analogous to optimal merge/Huffman order. It keeps intermediate sizes balanced and works especially well with the quadratic base case in the shared helper.

For bounds $1, 2, 3$, the factors are

$$(1+x)(1+x+x^2)(1+x+x^2+x^3).$$

The coefficient of x^4 is five, corresponding to $(0, 1, 3), (0, 2, 2), (1, 0, 3), (1, 1, 2), (1, 2, 1)$.

3.3 Correctness proof

Lemma 1. The coefficient of degree s after multiplying any subset of factors equals the number of settings of those dials with total s .

Proof. Induct on factors. Multiplication chooses one exponent from the new factor and convolves it with every previous total, exactly enumerating all extended settings once. QED.

Lemma 2. Truncation above K cannot change coefficient K later.

Proof. Every future factor has only nonnegative exponents. Multiplication can only preserve or increase a term's degree. QED.

The final polynomial therefore has the desired count at degree K , regardless of the associative multiplication order used by the heap.

3.4 Complexity, pitfalls, and tests

Balanced NTT multiplication is quasi-linear in the total retained product size, with logarithmic merge levels; a safe course bound is $O(K \log K \log n)$ time and $O(K+n)$ live coefficient storage, subject to the input's total-factor-size bound. Test target zero, zero-width dials, target larger than the sum of all bounds, and compare with bounded-knapsack DP.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
#include "../ntt.hpp"
using namespace std;
using course_ntt::convolution;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, target;
    if (!(cin >> n >> target)) return 0;

    struct Compare {
        bool operator()(const vector<int> &a, const vector<int> &b) const {
            return a.size() > b.size();
        }
    };
    priority_queue<vector<int>, vector<vector<int>>, Compare> products;
    for (int i = 0, bound; i < n; ++i) {
        cin >> bound;
        if (bound > 0) products.push(vector<int>(min(bound, target) + 1, 1));
    }
    if (products.empty()) {
        cout << (target == 0) << '\n';
        return 0;
    }
    while (products.size() > 1) {
        auto left = products.top(); products.pop();
        auto right = products.top(); products.pop();
        products.push(convolution(left, right, target + 1));
    }
    const auto &answer = products.top();
    cout << (target < static_cast<int>(answer.size())) ? answer[target] : 0 << '\n';
}
```

3.6 Full Python solution

```

import heapq
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
from ntt import convolution

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, target = data[:2]
    heap: list[tuple[int, int, list[int]]] = []
    serial = 0
    for bound in data[2 : 2 + n]:
        if bound > 0:
            polynomial = [1] * (min(bound, target) + 1)
            heapq.heappush(heap, (len(polynomial), serial, polynomial))
            serial += 1
    if not heap:
        print(int(target == 0))
        return
    while len(heap) > 1:
        _, _, left = heapq.heappop(heap)
        _, _, right = heapq.heappop(heap)
        product = convolution(left, right, target + 1)
        heapq.heappush(heap, (len(product), serial, product))
        serial += 1
    answer = heap[0][2]
    print(answer[target] if target < len(answer) else 0)

if __name__ == "__main__":
    main()

```

4 C. Huge Linear Recurrence

4.1 Replace matrix exponentiation by polynomial reduction

A companion matrix would give $O(k^3 \log n)$ with ordinary multiplication. We only need the first row action, and the characteristic relation lets us keep just k coefficients.

Represent a reduced polynomial by coefficients of $1, x, \dots, x^{(k-1)}$. To multiply two representations:

1. form their ordinary product, of degree at most $2k-2$;
2. for degrees from high to low, replace x^d by $c[0]x^{(d-1)} + \dots + c[k-1]x^{(d-k)}$;
3. retain the first k coefficients.

Binary exponentiation starts with polynomial one and base polynomial x . For order one, x immediately reduces to $c[0]$.

For Fibonacci, the relation is $x^2 = x + 1$. Reducing x^{10} yields coefficients $34 + 55x$, hence $F[10] = 34 \cdot F[0] + 55 \cdot F[1] = 55$.

4.2 Correctness proof

Lemma 1. `combine` returns the product of its inputs modulo the characteristic relation.

Proof. The ordinary product is exact. Descending reduction applies the defining equality to every term of degree at least k ; descending order also reduces any terms newly moved to lower but still excessive degrees. The final polynomial is congruent to the product and has degree below k . QED.

Lemma 2. After each exponentiation step, `result * baseremaining` is congruent to the original x^n .

Proof. This is the standard binary-exponentiation invariant, with every product replaced by an equivalent reduced product using Lemma 1. QED.

Theorem. The final dot product equals $a[n]$.

Proof. The reduced polynomial is congruent to x^n . The recurrence makes the same characteristic relation valid for sequence shifts, so replacing each x^i by $a[i]$ yields the stated linear combination for $a[n]$. QED.

4.3 Complexity, pitfalls, and tests

Time is $O(k^2 \log n)$ and memory is $O(k)$, excluding the temporary $O(k)$ product. Test $n < k$, order one, zero coefficients, and compare against a direct recurrence for small n .

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int64 MOD = 998244353;

vector<int64> combine(
    const vector<int64> &left,
    const vector<int64> &right,
    const vector<int64> &coefficient
) {
    int order = coefficient.size();
    vector<int64> product(2 * order - 1);
    for (int i = 0; i < order; ++i)
        for (int j = 0; j < order; ++j)
            product[i + j] = (product[i + j] + left[i] * right[j]) % MOD;
    for (int degree = 2 * order - 2; degree >= order; --degree)
        for (int back = 1; back <= order; ++back)
            product[degree - back] = (
                product[degree - back] + product[degree] * coefficient[back - 1]
            ) % MOD;
    product.resize(order);
    return product;
}

int64 nth_term(
    const vector<int64> &initial,
    const vector<int64> &coefficient,
    unsigned long long index
) {
    int order = initial.size();
    if (index < static_cast<unsigned long long>(order)) return initial[index];
    vector<int64> result(order), base(order);
    result[0] = 1;
    if (order == 1)
        base[0] = coefficient[0];
    else
        base[1] = 1;
    while (index > 0) {
        if (index & 1) result = combine(result, base, coefficient);
        base = combine(base, base, coefficient);
        index >>= 1;
    }
    int64 answer = 0;
    for (int i = 0; i < order; ++i)
        answer = (answer + result[i] * initial[i]) % MOD;
    return answer;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int order;
    unsigned long long index;
    if (!(cin >> order >> index)) return 0;
    vector<int64> initial(order), coefficient(order);
    for (int64 &value : initial) cin >> value;
    for (int64 &value : coefficient) cin >> value;
    cout << nth_term(initial, coefficient, index) << '\n';
}

```

4.5 Full Python solution

```

import sys

MOD = 998_244_353

def combine(left: list[int], right: list[int], coefficient: list[int]) -> list[int]:
    order = len(coefficient)
    product = [0] * (2 * order - 1)
    for i, a in enumerate(left):
        for j, b in enumerate(right):
            product[i + j] = (product[i + j] + a * b) % MOD
    for degree in range(2 * order - 2, order - 1, -1):
        value = product[degree]
        for back, coefficient_value in enumerate(coefficient, 1):
            product[degree - back] = (
                product[degree - back] + value * coefficient_value
            ) % MOD
    return product[:order]

def nth_term(initial: list[int], coefficient: list[int], index: int) -> int:
    order = len(initial)
    if index < order:
        return initial[index]
    result = [1] + [0] * (order - 1)
    base = [coefficient[0]] if order == 1 else [0, 1] + [0] * (order - 2)
    while index:
        if index & 1:
            result = combine(result, base, coefficient)
            base = combine(base, base, coefficient)
            index >>= 1
    return sum(a * b for a, b in zip(result, initial)) % MOD

```

```
def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    order, index = data[:2]
    initial = data[2 : 2 + order]
    coefficient = data[2 + order : 2 + 2 * order]
    print(nth_term(initial, coefficient, index))

if __name__ == "__main__":
    main()
```

5 D. Recurrence Recovery

5.1 What a discrepancy tells us

Let the current relation be stored as

$$C[0]s[n] + C[1]s[n-1] + \dots = 0, \text{ with } C[0]=1.$$

Its discrepancy at the newest position is that left side. A zero discrepancy means the relation still works. Otherwise, recall an older relation B whose nonzero discrepancy was b . After shifting it to align with the current position, subtracting $\text{discrepancy}/b$ copies cancels the new error.

Sometimes this repair needs a longer relation. The condition $2*L \leq \text{position}$ detects that the old length can no longer explain the processed prefix; the previous current relation becomes the new saved baseline, and the minimal possible length becomes $\text{position}+1-L$.

For Fibonacci observations, the algorithm recovers coefficients $1, 1$:

$$s[n] = s[n-1] + s[n-2].$$

Problem C 's evaluator then computes the distant term.

5.2 Correctness argument

The full Berlekamp–Massey theorem states that after each processed position, C is a shortest linear relation for that prefix. The update cancels the only new violated equation by a shifted relation known to have a nonzero pivot discrepancy. When the current length is still sufficient, no increase is needed. When $2L \leq \text{position}$, any relation satisfying both the previous prefix and the new equation needs length at least $\text{position}+1-L$; the update attains that bound. Induction establishes minimality.

The statement guarantees enough observations ($2r \leq L$) to identify the true minimal recurrence. Once recovered, the proof of C shows that polynomial exponentiation returns the required future term. The all-zero sequence never has a discrepancy, so its returned order is zero and every future term is zero.

5.3 Complexity, pitfalls, and tests

Recovery costs $O(L^2)$ in the worst case. Evaluation costs $O(r^2 \log n)$. Memory is $O(L+r)$. Test all zeros, a geometric sequence, Fibonacci, sums of several distinct geometric sequences, and indices already inside the observed prefix.

5.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int64 MOD = 998244353;

int64 power(int64 base, int64 exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1) result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}

vector<int64> berlekamp_massey(const vector<int64> &sequence) {
    vector<int64> current{1}, saved{1};
    int length = 0, shift = 1;
    int64 saved_discrepancy = 1;
    for (int position = 0; position < static_cast<int>(sequence.size()); ++position) {
        int64 discrepancy = sequence[position];
        for (int i = 1; i <= length; ++i)
            discrepancy = (discrepancy + current[i] * sequence[position - i]) % MOD;
        if (discrepancy == 0) {
            ++shift;
            continue;
        }
        vector<int64> previous = current;
        int64 scale = discrepancy * power(saved_discrepancy, MOD - 2) % MOD;
        if (current.size() < saved.size() + shift)
            current.resize(saved.size() + shift);
        for (int i = 0; i < static_cast<int>(saved.size()); ++i) {
            current[i + shift] = (current[i + shift] - scale * saved[i]) % MOD;
            if (current[i + shift] < 0) current[i + shift] += MOD;
        }
        if (2 * length <= position) {
            length = position + 1 - length;
            saved = move(previous);
            saved_discrepancy = discrepancy;
            shift = 1;
        } else {
            ++shift;
        }
    }
    current.resize(length + 1);
    vector<int64> recurrence(length);
    for (int i = 0; i < length; ++i)
```

```

        recurrence[i] = (MOD - current[i + 1]) % MOD;
    return recurrence;
}

vector<int64> combine(const vector<int64> &a, const vector<int64> &b,
                    const vector<int64> &recurrence) {
    int order = recurrence.size();
    vector<int64> product(2 * order - 1);
    for (int i = 0; i < order; ++i)
        for (int j = 0; j < order; ++j)
            product[i + j] = (product[i + j] + a[i] * b[j]) % MOD;
    for (int degree = 2 * order - 2; degree >= order; --degree)
        for (int back = 1; back <= order; ++back)
            product[degree - back] = (
                product[degree - back] + product[degree] * recurrence[back - 1]
            ) % MOD;
    product.resize(order);
    return product;
}

int64 nth_term(const vector<int64> &sequence, const vector<int64> &recurrence,
               unsigned long long index) {
    if (index < sequence.size()) return sequence[index];
    int order = recurrence.size();
    if (order == 0) return 0;
    vector<int64> result(order), base(order);
    result[0] = 1;
    if (order == 1) base[0] = recurrence[0];
    else base[1] = 1;
    while (index) {
        if (index & 1) result = combine(result, base, recurrence);
        base = combine(base, base, recurrence);
        index >>= 1;
    }
    int64 answer = 0;
    for (int i = 0; i < order; ++i)
        answer = (answer + result[i] * sequence[i]) % MOD;
    return answer;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int length;
    unsigned long long index;
    if (!(cin >> length >> index)) return 0;
    vector<int64> sequence(length);
    for (int64 &value : sequence) cin >> value;
    vector<int64> recurrence = berlekamp_massey(sequence);
    cout << recurrence.size() << ' ' << nth_term(sequence, recurrence, index) << '\n';
}

```

5.5 Full Python solution

```
import sys

MOD = 998_244_353

def berlekamp_massey(sequence: list[int]) -> list[int]:
    current = [1]
    saved = [1]
    length = 0
    shift = 1
    saved_discrepancy = 1
    for position, term in enumerate(sequence):
        discrepancy = term
        for i in range(1, length + 1):
            discrepancy = (discrepancy + current[i] * sequence[position - i]) % MOD
        if discrepancy == 0:
            shift += 1
            continue
        previous = current[:]
        scale = discrepancy * pow(saved_discrepancy, MOD - 2, MOD) % MOD
        current += [0] * max(0, len(saved) + shift - len(current))
        for i, value in enumerate(saved):
            current[i + shift] = (current[i + shift] - scale * value) % MOD
        if 2 * length <= position:
            length = position + 1 - length
            saved = previous
            saved_discrepancy = discrepancy
            shift = 1
        else:
            shift += 1
    return [(-current[i]) % MOD for i in range(1, length + 1)]

def combine(left: list[int], right: list[int], recurrence: list[int]) -> list[int]:
    order = len(recurrence)
    product = [0] * (2 * order - 1)
    for i, a in enumerate(left):
        for j, b in enumerate(right):
            product[i + j] = (product[i + j] + a * b) % MOD
    for degree in range(2 * order - 2, order - 1, -1):
        value = product[degree]
        for back, coefficient in enumerate(recurrence, 1):
            product[degree - back] = (
                product[degree - back] + value * coefficient
            ) % MOD
    return product[:order]
```

```

def nth_term(sequence: list[int], recurrence: list[int], index: int) -> int:
    if index < len(sequence):
        return sequence[index]
    order = len(recurrence)
    if order == 0:
        return 0
    result = [1] + [0] * (order - 1)
    base = [recurrence[0]] if order == 1 else [0, 1] + [0] * (order - 2)
    while index:
        if index & 1:
            result = combine(result, base, recurrence)
            base = combine(base, base, recurrence)
            index >>= 1
    return sum(result[i] * sequence[i] for i in range(order)) % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    length, index = data[:2]
    sequence = data[2 : 2 + length]
    recurrence = berlekamp_massey(sequence)
    print(len(recurrence), nth_term(sequence, recurrence, index))

if __name__ == "__main__":
    main()

```

6 Stress-testing strategy

A uses explicit shifts. B uses ordinary bounded-knapsack DP. C extends the recurrence sequentially. D's generator uses sums of distinct geometric sequences: the minimal order is independently known to equal the number of distinct nonzero bases, while the distant term is evaluated directly by modular power. These oracles test the models, not just the implementation syntax.

7 What to carry forward

Write the coefficient equation before reaching for an FFT. Reverse arrays only after deriving the desired index sum. For huge recurrence indices, reduce polynomials instead of exponentiating a general matrix. When a recurrence is hidden, a nonzero prediction error is useful algebraic information, not merely a failure.